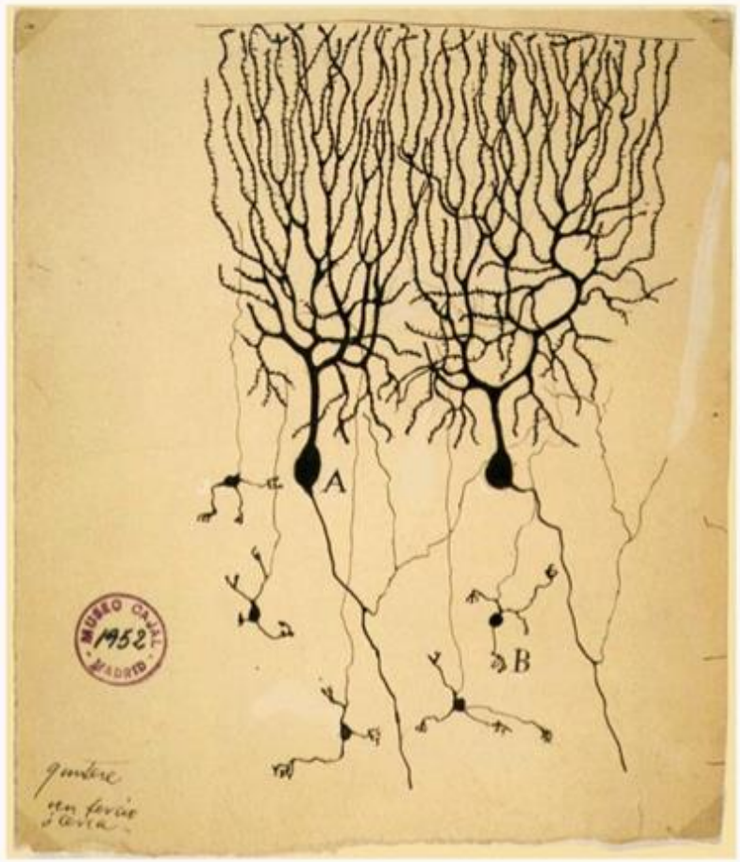
Machine Learning CS 4641-7641

Neural Networks Forward Pass and Back Propagation

Nimisha Roy
Georgia Tech

Slides adapted from Mahdi Roozbahani

Inspiration from Biological Neurons



The first drawing of a brain cells by Santiago Ramón y Cajal in 1899

Neurons: core components of brain and the nervous system consisting of

1. Dendrites that collect information from other neurons
2. An axon that generates outgoing spikes

Georgia Tech

Neural Networks

- A robust approach for approximating **real-valued, discrete-valued or vector valued** functions

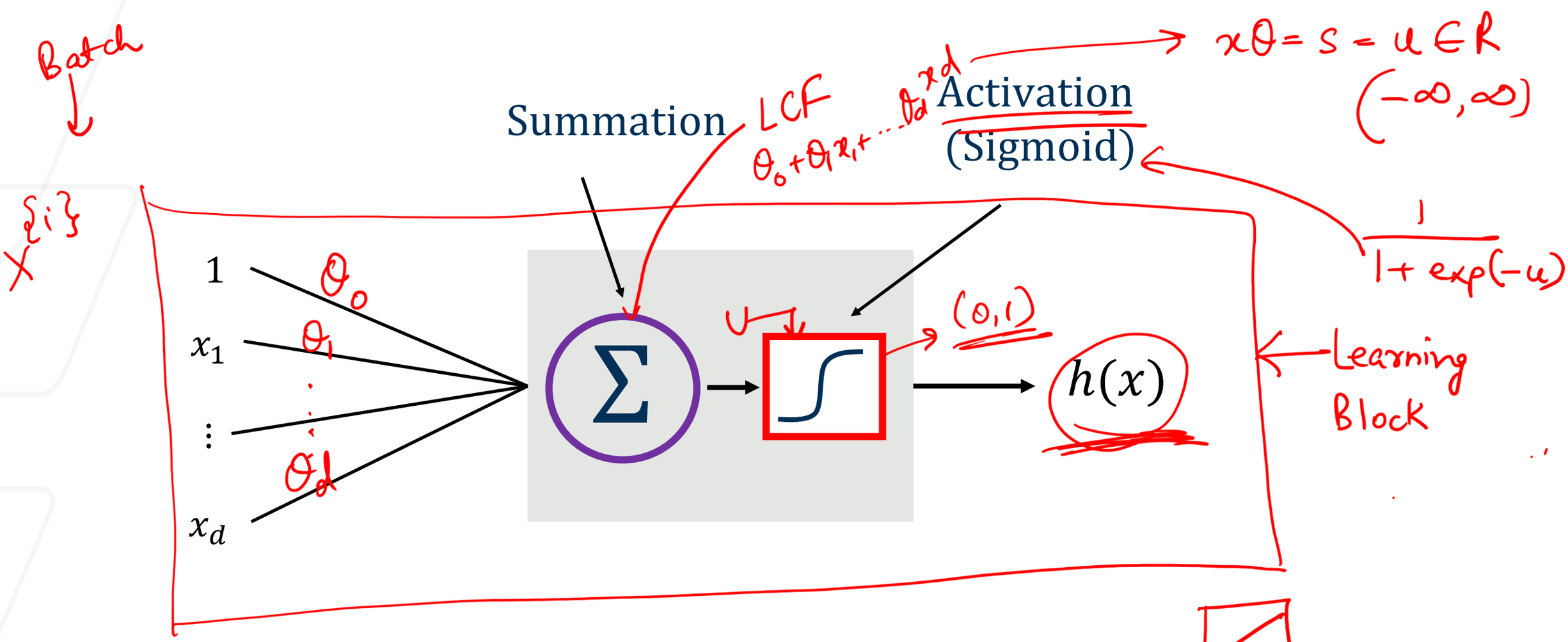
- Among the most effective **general purpose** supervised learning methods

- Apt at modeling **complex and hard to interpret data** such as images and audio

- The **backpropagation algorithm** for neural networks has been shown successful in many practical problems:

  - Handwritten character recognition, speech recognition, object recognition, some NLP problems

Georgia Tech



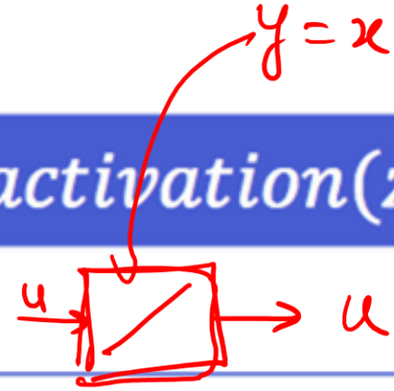
$$output = activation(x\theta + b)$$

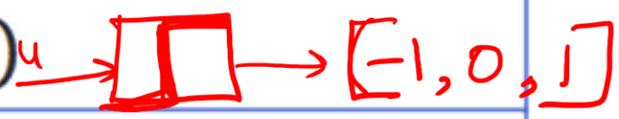
$$h(x) = o(x\theta = u) \rightarrow \text{logistic regression}$$

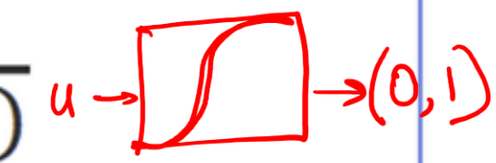
$x\theta$ = LCF = u

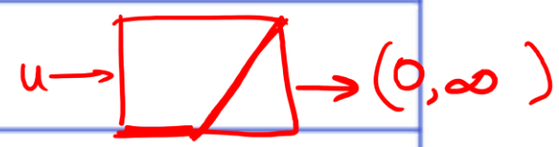
Activation Function → Introduce some kind of non-linearity in our model

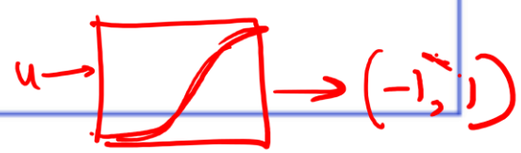
Name of the neuron	Activation function: $activation(z)$
Linear unit	z
Threshold/sign unit	$\text{sgn}(z)$
Sigmoid unit	$\frac{1}{1+\exp(-z)}$
Rectified linear unit (ReLU)	$\max(0, z)$
Tanh unit	$\tanh(z)$











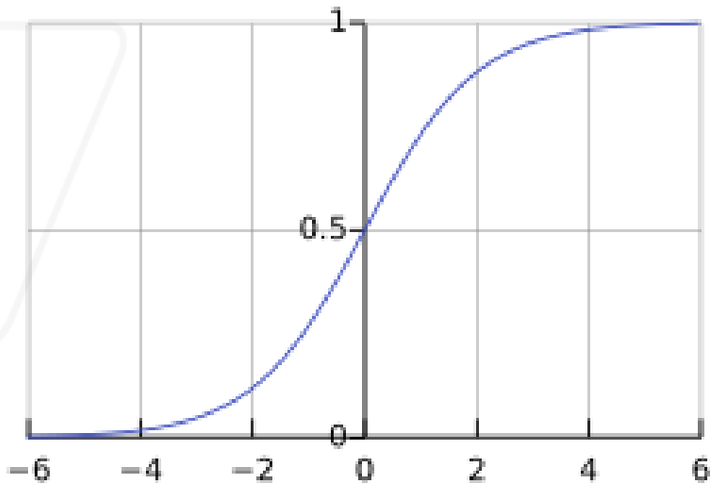
Activation function should introduce non-linearity

Very very Popular

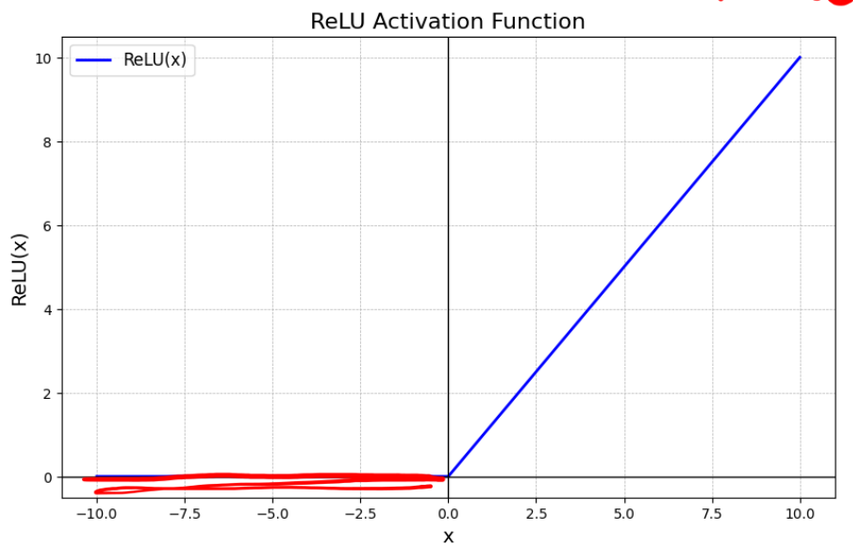
- simple
- converge very quickly

Different activation functions: o

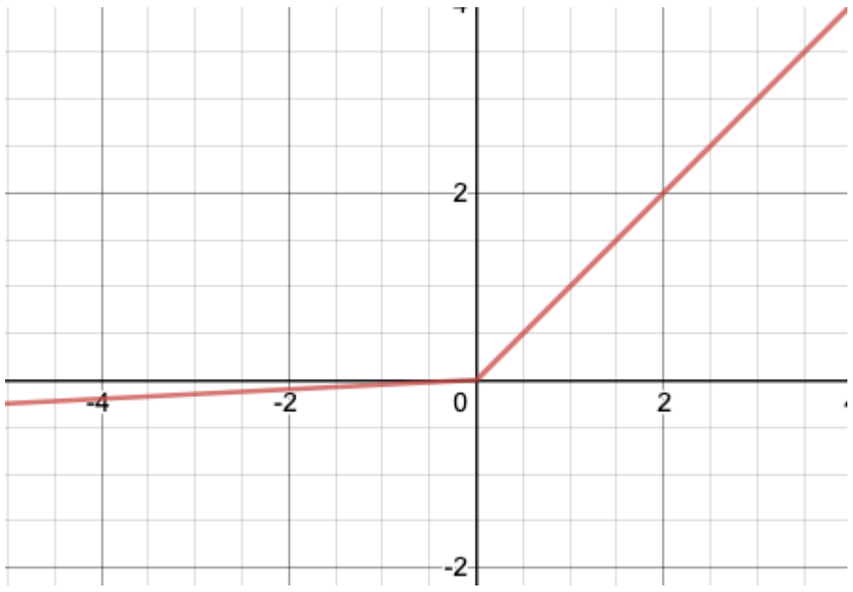
Sigmoid



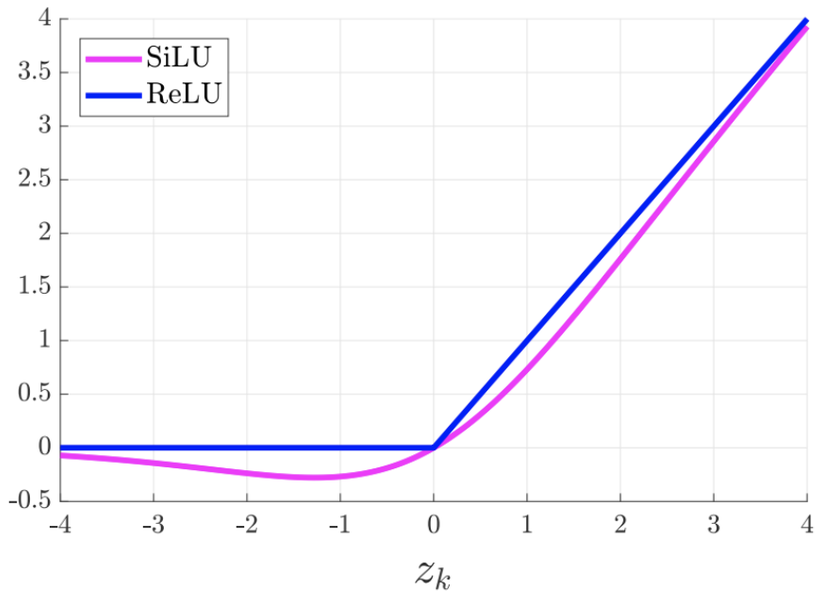
ReLu



Leaky ReLu

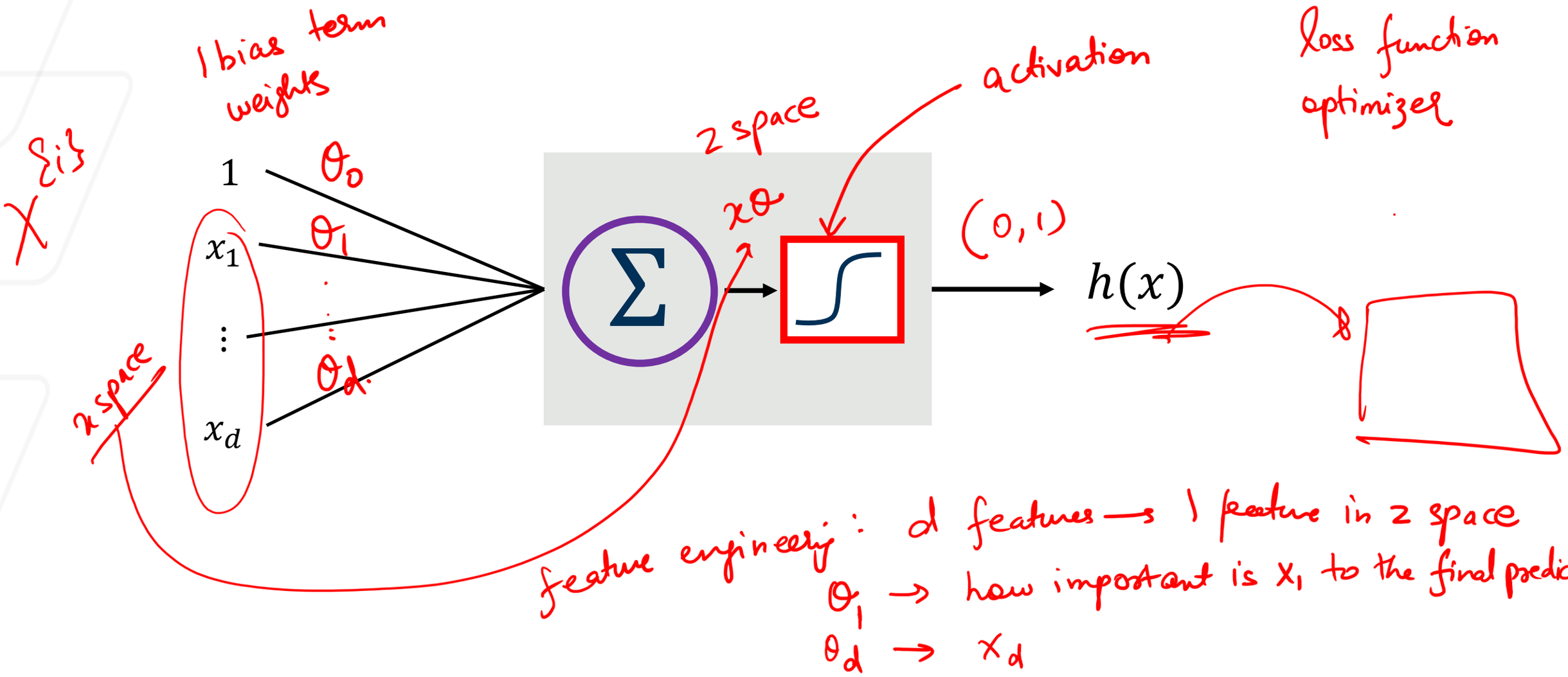


SiLu = Sigmoid + ReLu



Georgia Tech

In Neural Network, 1 learning block is 1 building block



A **neural network** is essentially a **stack (or composition)** of many **learning blocks**, where:

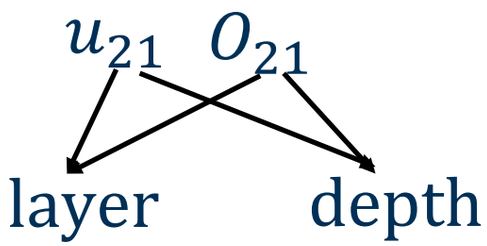
$$\textit{Output of Block } i \;=\; \textit{Input to Block } (i+1)$$

Building a Network: NN Regression

$u_{11} = \theta_0 \cdot 1 + \theta_2 \cdot x_i$

$u_{12} = \theta_1 + \theta_3 \cdot x_i$

2 building blocks



$u_{21} = \theta_4 \cdot 1 + \theta_5 \cdot O_{11} + \theta_6 \cdot O_{12}$

Input layer

1st hidden layer

Output layer

Neuron or node

feature x_i

θ_0

θ_1

θ_2

θ_3

weight

Summation

u_{11}

u_{12}

Summation output

$\frac{1}{1+\exp(-u)}$

Activation (Sigmoid)

Bias

1

O_{11}

O_{12}

(0, 1)

Sigmoid output

(0,1)

θ_4

θ_5

θ_6

2nd input layer
z space

u_{21}

Regression

O_{21}

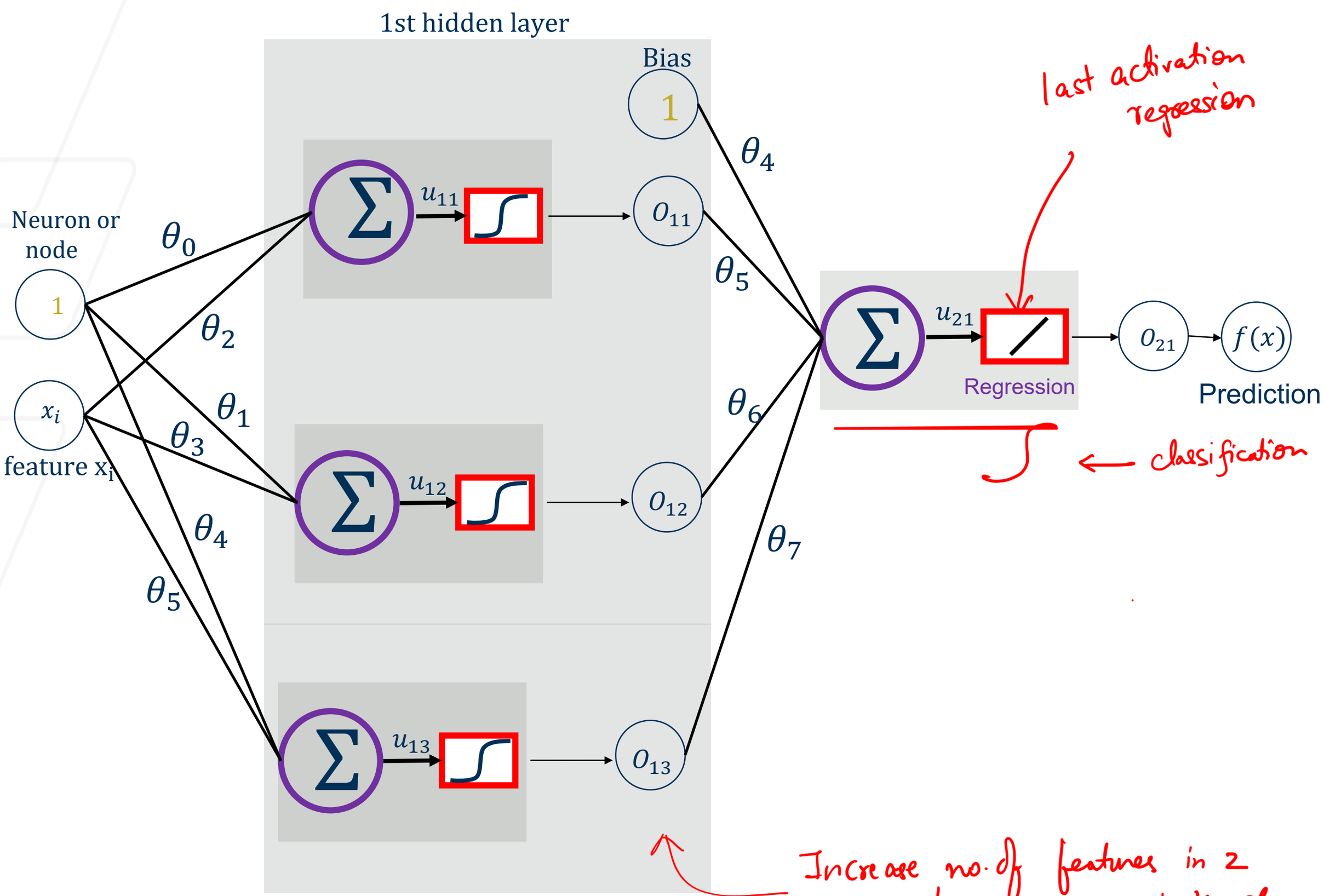
$f(x)$

Prediction

$O_{21} = u_{21} = f(z)$

input features are always fully connected to the first layer

Georgia Tech


1st hidden layer
Bias
1
Neuron or node
1
x_i
feature x_i
θ_0
θ_1
θ_2
θ_3
θ_4
θ_5
θ_4
θ_5
θ_6
θ_7
u_{11}
u_{12}
u_{13}
o_{11}
o_{12}
o_{13}
u_{21}
o_{21}
$f(x)$
Regression
Prediction
last activation regression
classification
Increase no. of features in 2 space by increasing depth of the previous layer

1st hidden layer

2nd hidden layer

Bias

Bias

Neuron or node

feature x_i

1

x_i

u_{11}

u_{12}

u_{13}

u_{21}

u_{22}

u_{23}

O_{11}

O_{12}

O_{13}

O_{21}

O_{22}

O_{23}

1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22

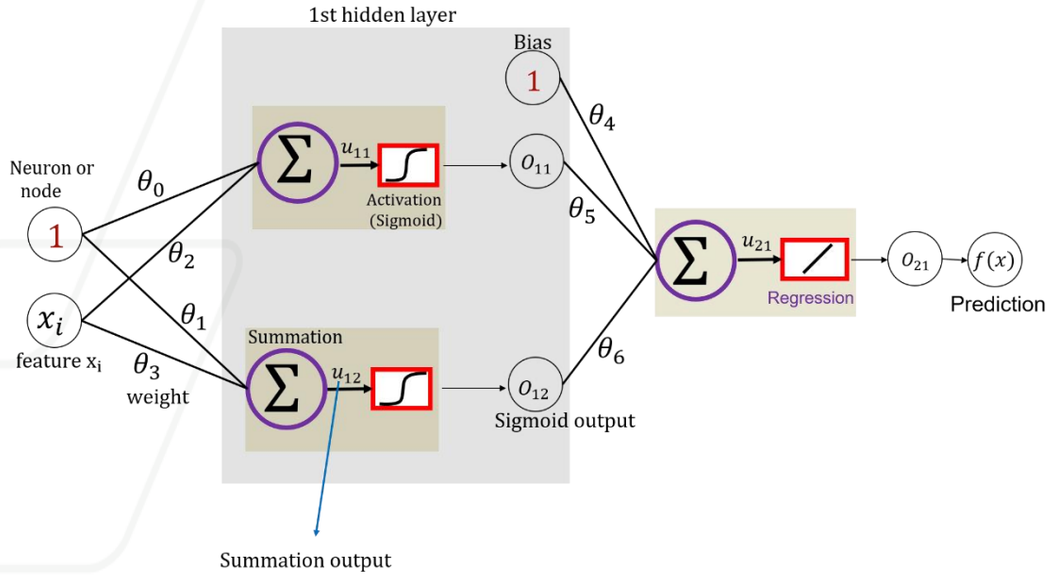
$2 \times 3 = 6$

$4 \times 3 = 12$

$4 \times 1 = 4$

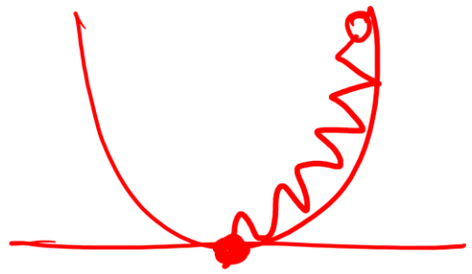
22 parameters

Georgia Tech



Gradient Descent

$$\theta^{t+1} = \theta^{t} - \alpha \cdot \frac{\partial L}{\partial \theta}$$



Step0 : Initialize θ's. Do not initialize θ to 0.

Step1 : Calculate u's & o's.
(u's ↓ θ, o's ↓ activation function)

FORWARD PASS

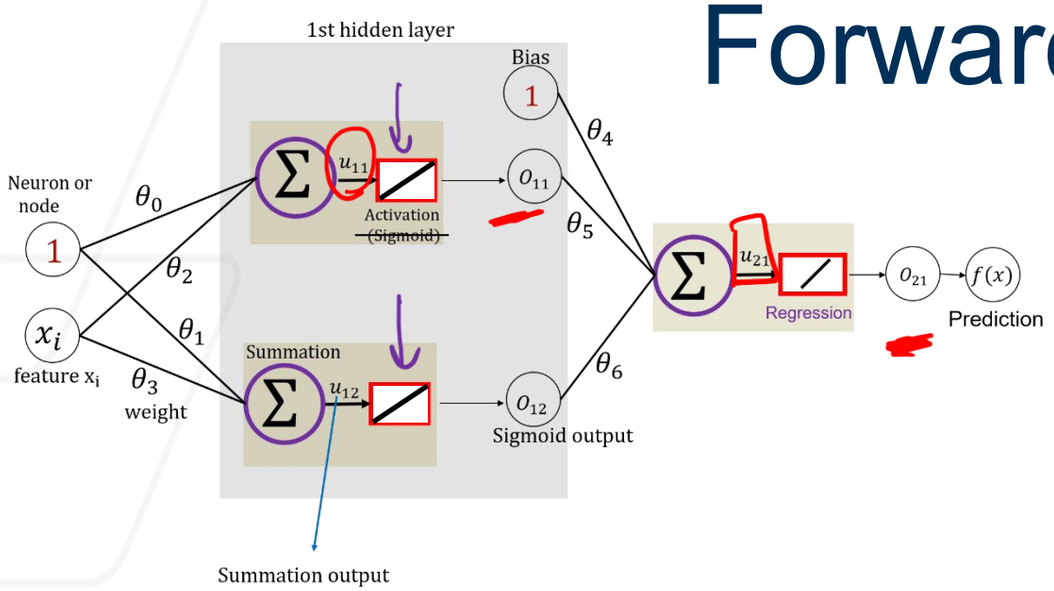
Step3 : Recalculate θ^{t+1}.

BACK PROPAGATION

Repeat Until Convergence.

Regression — MSE
Classification — CE
— CE with softmax

Forward Propagation: Rigid Network



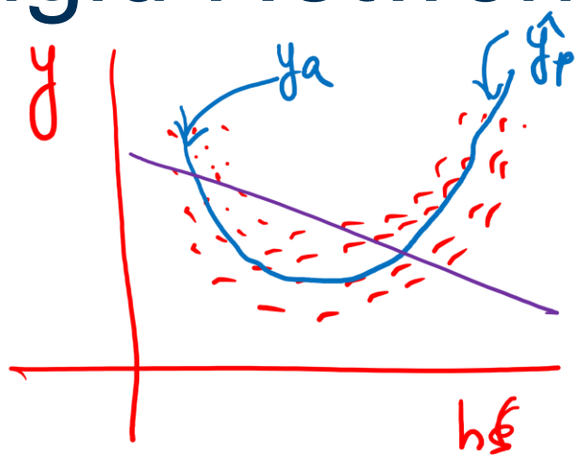
$$u_{11} = \theta_0 + \theta_2 x_i$$

$$o_{11} = u_{11}$$

$$u_{12} = \theta_1 + \theta_3 x_i$$

u_{12}: 1st layer, 2nd block

$$o_{12} = u_{12}$$



$$u_{21} = \theta_4 + \theta_5 \cdot o_{11} + o_{12} \cdot \theta_6$$

$$= \theta_4 + \theta_5(\theta_0 + \theta_2 x_i) + \theta_6(\theta_1 + \theta_3 x_i)$$

$$u_{21} = (\theta_4 + \theta_5\theta_0 + \theta_6\theta_1) + x_i(\theta_5\theta_2 + \theta_6\theta_3)$$

$$o_{21} = u_{21} = f(x)$$

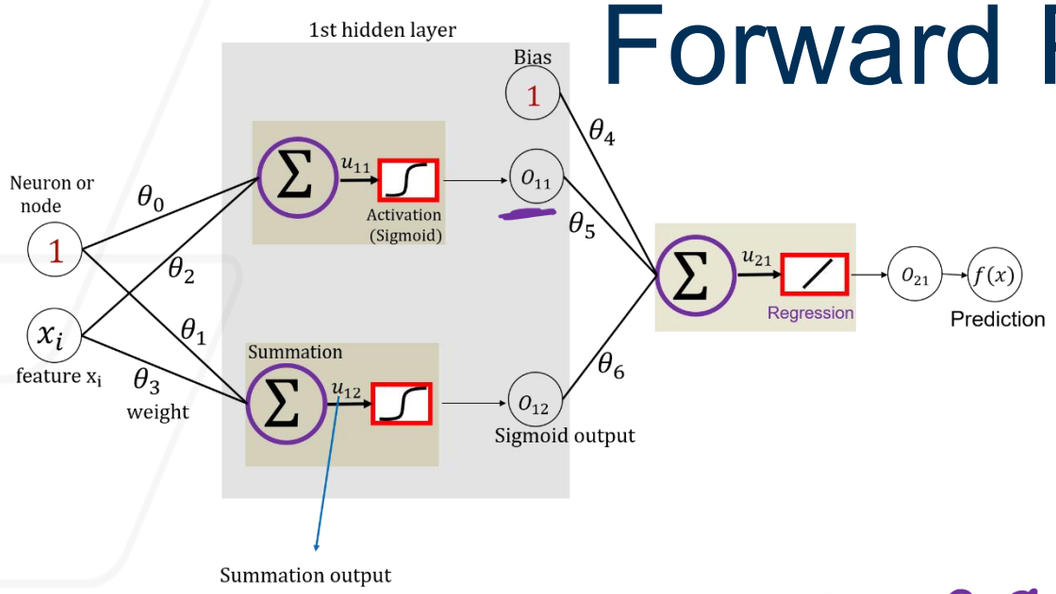
$\theta_0 + x_i \theta_1$

θ_1: slope or rotation

θ_0: y intercept

Georgia Tech

Forward Propagation: Flexible Network



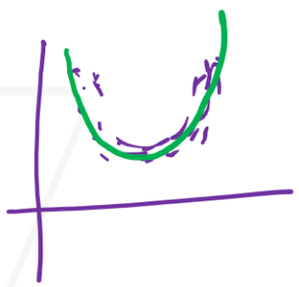
$$u_{11} = \theta_0 + \theta_2 x_i$$

$$o_{11} = \frac{1}{1 + \exp(-u_{11})} = \frac{1}{1 + \exp(-\theta_0 - \theta_2 x_i)}$$

horizontal shrinking or stretching

horizontal translation

0.5



$$u_{12} = \theta_1 + \theta_3 x_i$$

$$o_{12} = \frac{1}{1 + \exp(-\theta_1 - \theta_3 x_i)}$$

$$u_{21} = \theta_4 + \theta_5 o_{11} + \theta_6 o_{12}$$

$$= \theta_4 + \frac{\theta_5}{1 + \exp(-\theta_0 - \theta_2 x_i)} + \frac{\theta_6}{1 + \exp(-\theta_1 - \theta_3 x_i)}$$

$$o_{21} = f(x) = u_{21}$$

vertical translation

vertical shrinky or stretchy

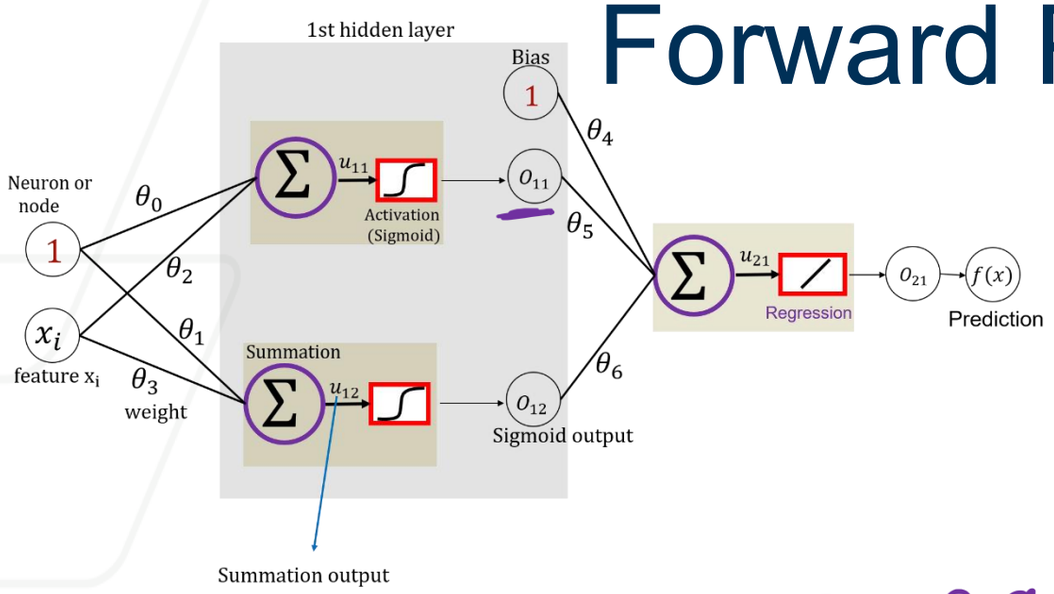
horizontal translation

horizontal shrinking or stretchy

Georgia Tech

https://playground.tensorflow.org

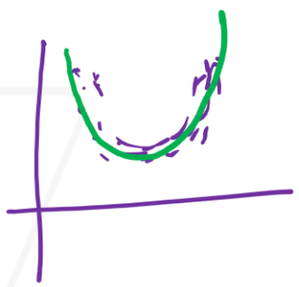
Forward Propagation: Flexible Network



$$u_{11} = \theta_0 + \theta_2 x_i$$

$$o_{11} = \frac{1}{1 + \exp(-u_{11})} = \frac{1}{1 + \exp(-\theta_0 - \theta_2 x_i)}$$

horizontal shrinking or stretching



$$u_{12} = \theta_1 + \theta_3 x_i$$

$$o_{12} = \frac{1}{1 + \exp(-\theta_1 - \theta_3 x_i)}$$

$$u_{21} = \theta_4 + \theta_5 o_{11} + \theta_6 o_{12}$$

$$= \theta_4 + \frac{\theta_5}{1 + \exp(-\theta_0 - \theta_2 x_i)} + \frac{\theta_6}{1 + \exp(-\theta_1 - \theta_3 x_i)}$$

$$o_{21} = f(x) = u_{21}$$

vertical translation

vertical shrinking or stretching

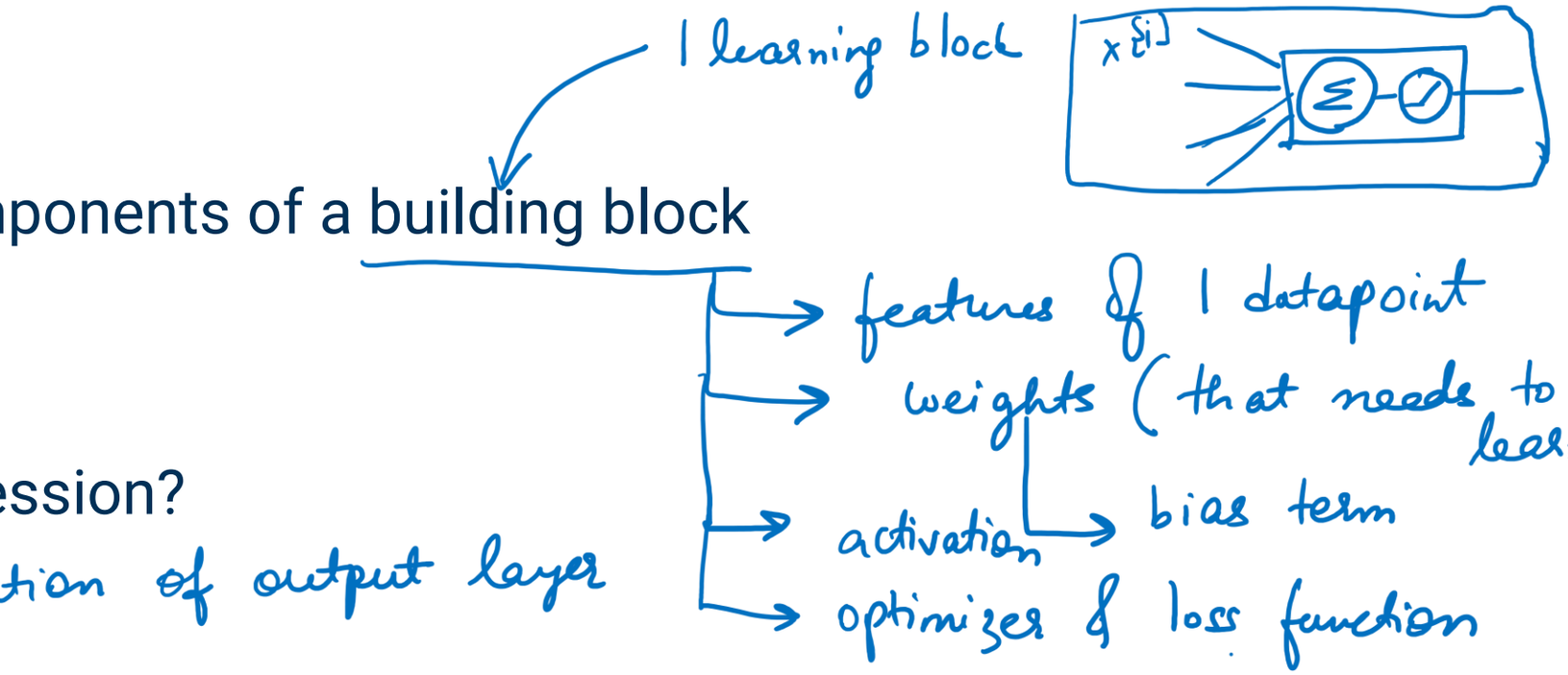
horizontal translation

horizontal shrinking or stretching

Georgia Tech

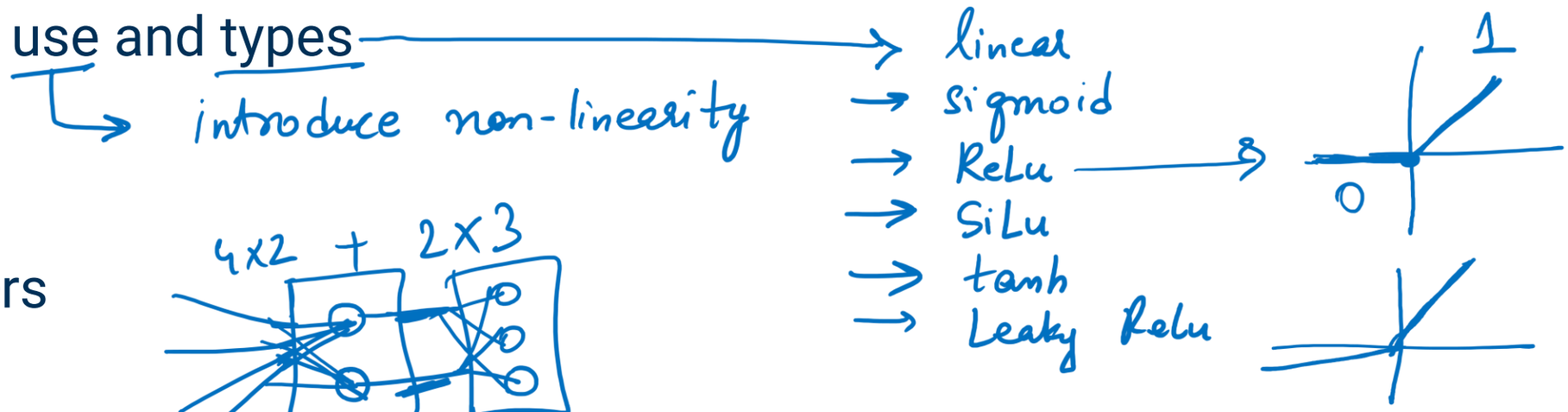
Recap

- Neural Network – components of a building block



- Classification or Regression?

- activation function – use and types



- Number of parameters

- GD Step 0 and Step 1

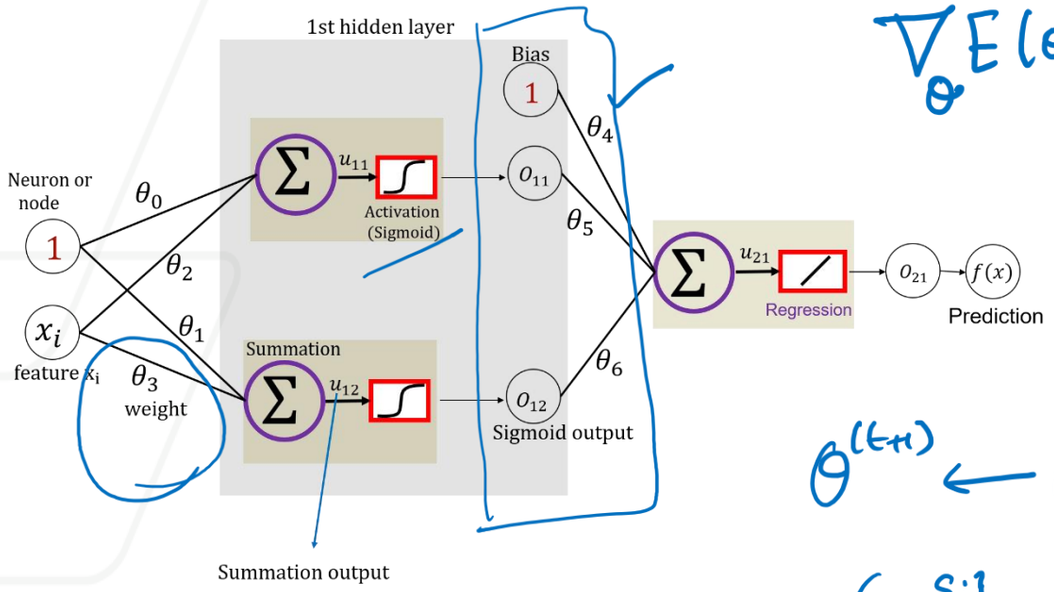
Step 0: Initialize all parameters/weights

Step 1: Forward Pass → Compute u's, Compute o's

Step 2: Back Propagate

Georgia Tech

Backward Propagation



$\nabla_\theta E(\theta)$

Objective Function $E(\theta)$: min. $\frac{1}{2N}\sum_{i=1}^{n}\left(y_a^{\{i\}} - f(x)^{\{i\}}\right)^2$

1 datapoint

$$E(\theta) = \frac{1}{2}\left(y_a^{\{i\}} - f(x)^{\{i\}}\right)^2$$

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \alpha \cdot \nabla_\theta E(\theta)$$

$$\nabla_\theta E(\theta) = \left(y_a^{\{i\}} - f(x)^{\{i\}}\right)\frac{\partial f(x)}{\partial \theta} = \Delta \frac{\partial f(x)}{\partial \theta}$$

$$\nabla_{\theta_0} E(\theta), \nabla_{\theta_1} E(\theta), \ldots\ldots, \nabla_{\theta_6} E(\theta)$$

$$f(x) = \theta_4 + \theta_5 o_{11} + \theta_6 o_{12}$$

$$\theta_4^{(t+1)} \leftarrow \theta_4^{(t)} - \alpha \cdot \Delta \cdot \frac{\partial f(x)}{\partial \theta_4}$$

$$\frac{\partial f(x)}{\partial \theta_4} = 1$$

$$\theta_4^{(t+1)} \leftarrow \theta_4^{(t)} - \alpha \Delta \cdot 1$$

$$\frac{\partial f(x)}{\partial \theta_5} = o_{11}$$

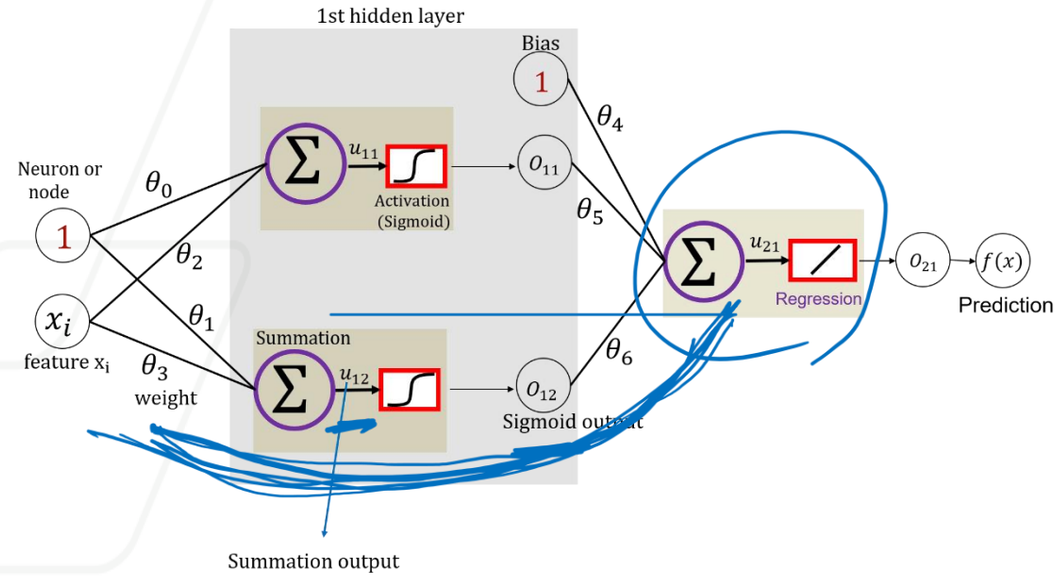
$$\theta_5^{(t+1)} \leftarrow \theta_5^{(t)} - \alpha \cdot \Delta \cdot o_{11}$$

$$\frac{\partial f(x)}{\partial \theta_6} = o_{12}$$

$$\theta_6^{(t+1)} \leftarrow \theta_6^{(t)} - \alpha \Delta o_{12}$$

Georgia Tech

Backward Propagation



Objective Function $E(\theta)$:

Chain Rule
Stop at all o's & all u's

$$\frac{\partial f(x)}{\partial \theta_3}$$

$$\frac{\partial f(x)}{\partial o_{12}} \times \frac{\partial o_{12}}{\partial u_{12}} \times \frac{\partial u_{12}}{\partial \theta_3}$$

$\frac{\partial f(x)}{\partial o_{12}} \rightarrow \theta_6$

$\frac{\partial o_{12}}{\partial u_{12}} \rightarrow o_{12}[1 - o_{12}]$

$\frac{\partial u_{12}}{\partial \theta_3} \rightarrow x_1$

Sigmoid

$$f(x) = \theta_4 + \theta_5 o_{11} + \theta_6 o_{12}$$

$$u_{12} = \theta_1 + \theta_3 x_1$$

$$o = \frac{1}{1+\exp(-u)} = (1+\exp(-u))^{-1}$$

$$\frac{\partial o}{\partial u} = -1 \cdot (1+\exp(-u))^{-2} \cdot (-\exp(-u))$$

$$= \frac{1+\exp(-u)-1}{(1+\exp(-u))^2} = \frac{1}{1+\exp(-u)}\left[\frac{1+\exp(-u)}{1+\exp(-u)} - \frac{1}{1+\exp(-u)}\right]$$

$(0,1)$

$$\frac{\partial o}{\partial u} = o[1-o]$$

$(0,1) \leq 0.25$

$$\theta_3^{(t+1)} \leftarrow \theta_3^{(t)} - \alpha \cdot \nabla_{\theta_3} E(\theta) \leftarrow \theta_3^{(t)} - \alpha \cdot \Delta \cdot \theta_6 \cdot x_1 \cdot o_{12}(1-o_{12})$$

Georgia Tech

Vanishing and Exploding Gradient

In deep networks, gradients can **shrink (vanish)** or **grow uncontrollably (explode)** during backpropagation, making training unstable or slow.

Sigmoid Activation

- Output range: (0, 1). Derivative: $\sigma'(x) = \sigma(x)(1 - \sigma(x)) \leq 0.25$
- Large |x| → output saturates near 0 or 1 → gradients ≈ 0. → **Vanishing gradient problem** → early layers stop learning

ReLU Activation

- Derivative: 1 for $x > 0$, 0 for $x \leq 0$
- Avoids vanishing (no upper saturation)
- But:
  - Large positive activations + large weights → gradients **amplify** across layers. → **Exploding gradients**
  - Many $x \leq 0$ → gradients = 0 → **Dead ReLUs**

Mitigation

- **BatchNorm** to stabilize activations - Normalizes layer outputs to have zero mean and unit variance before activation.
- **Gradient clipping** to limit large updates - puts an upper bound on gradient magnitude (e.g., clipping at 1.0).
- Use **ReLU variants** (Leaky RELU) – avoid dead ReLU.

Georgia Tech

Direction in Gradient Descent

- Each weight affects the total loss of the network.
- Backpropagation computes the **gradient** (magnitude and direction)— how much the loss changes if a weight changes.
- If the gradient is **positive** → decreasing the weight lowers the loss.
- If the gradient is **negative** → increasing the weight lowers the loss.
- Gradient Descent updates weights by taking small steps opposite to the gradient.
- This helps the model **learn** gradually and minimize error.

$$\theta^{(t+1)} \leftarrow \theta^{(t)} - \alpha \nabla_\theta E(\theta)$$

hyperparameter

Georgia Tech

Summary: Forward Pass, Backpropagation, and Gradient Descent

1 Forward Pass — Compute Outputs

- Inputs move through each layer of the network.
- Each layer applies its weights, bias, and activation function.
- The network produces predictions at the output layer.
- The difference between predictions and true values gives the **loss**.
- We compute all u’s and o’s.

2 Backpropagation — Compute Gradients

- The loss is sent backward through the network.
- Each layer calculates how much it contributed to the total error.
- We compute all gradients. Using the **chain rule**, gradients are propagated from output to input.
- This gives the “direction” each weight should move to reduce error.

3 Gradient Descent — Update Parameters

- The optimizer updates weights slightly in the opposite direction of the gradient.
- The size of the adjustment is controlled by the **learning rate**.
- Repeated passes gradually minimize the loss.
- *Model improves by iteratively reducing error over many epochs.*

Georgia Tech